

AI Engineering Intern Hiring Assignment

Smart Job Match Agent

Role: AI Engineer (Junior) | **Time Budget:** 6–8 hours | **Submission Deadline:** 24 hours from receipt

1. Overview

You are tasked with building a **Job Recommendation API** — an intelligent system that accepts a candidate's resume and returns a ranked list of relevant job openings, along with natural-language reasoning and a smart follow-up question to improve match quality.

This assignment is intentionally open-ended. We are not looking for perfection — we are looking for **how you think, how you make trade-offs, and how clearly you can explain your decisions.**

The system must combine:

- **Classical ML** (semantic embeddings + vector similarity) for initial ranking
 - **An LLM-powered agentic layer** (tool-calling) for reasoning and interaction
 - **A production-style REST API** that ties it all together
-

2. The Job Dataset

Use the dataset provided separately (`jobs.json` — 50 roles). Do **not** use any external jobs API or live data source. The focus is your system architecture, not data sourcing.

The dataset includes fields: `id`, `title`, `company`, `location`, `remote`, `skills`, `experience_years`, `salary_lpa`, `domain`, `description`.

Note: The dataset is intentionally diverse — spanning tech, finance, healthcare, legal, agritech, edtech, and more. A naive keyword-matching approach will perform poorly. This is by design.

3. What You Will Build

Part 1 — Semantic Similarity Ranking (*Classical ML*)

Embed both the job descriptions and the incoming resume using a sentence embedding model of your choice. Compute **cosine similarity** between the resume embedding and each job, and return a ranked list.

Requirements:

- Use any embedding model (e.g. `sentence-transformers`, OpenAI `text-embedding-*`, Cohere, etc.)
- Compute cosine similarity correctly
- Return the top-N ranked jobs with their similarity scores

You will be asked to justify your model choice in the write-up.

Part 2 — Agentic LLM Layer (*Tool Calling*)

Build a **two-step agent** using any LLM (GPT-4o, Claude, Gemini, Mistral, or any open-source model). The agent must use the LLM's **native function/tool calling API** — not a single monolithic prompt.

Step 1 – Resume Parser Tool

The agent calls a tool that extracts structured fields from the raw resume text:

```
{
  "name": "string",
  "skills": ["string"],
  "experience_years": "number",
  "preferred_roles": ["string"],
  "education": "string"
}
```

Step 2 – Match Reasoning Tool

The agent calls a tool that takes the top-5 similarity matches from Part 1 and returns a short natural-language explanation (2–3 sentences) for each job — covering *why* it is or isn't a strong fit for this specific candidate.

⚠ **Important:** The agent must make real, structured tool calls. Simulating tool calls inside a single prompt chain is not acceptable and will be evaluated accordingly.

Part 3 — Clarifying Question Generation (*Prompt Engineering*)

After producing the ranked list, the system must generate **one smart follow-up question** to ask the candidate — aimed at resolving an ambiguity or gap the agent noticed.

Examples of good questions:

- *"Several strong matches are remote-only — are you open to remote work?"*
- *"Your resume doesn't mention cloud platforms. Do you have hands-on AWS or GCP experience?"*
- *"Two of your top matches are in the finance domain — is that a preference, or would you consider other industries?"*

Requirements:

- The question must be **dynamically generated** by the LLM based on the specific resume and matched jobs
 - It must not be hardcoded or templated
 - It should be specific and useful — not generic like *"Tell me more about yourself"*
-

Part 4 — FastAPI REST API (*Production Code*)

Expose the full system as a REST API.

Mandatory endpoints:

```
POST /recommend
Content-Type: application/json

Request:
{
  "resume_text": "string"
}

Response:
{
  "candidate": {
    "name": "string",
    "skills": ["string"],
    "experience_years": number
  },
  "ranked_jobs": [
    {
      "id": number,
      "title": "string",
      "company": "string",
      "similarity_score": number,
      "explanation": "string"
    }
  ],
  "clarifying_question": "string"
}
```

Bonus endpoint:

```
POST /refine
Content-Type: application/json

Request:
{
  "resume_text": "string",
  "clarifying_question": "string",
  "candidate_answer": "string"
}

Response:
{
  "ranked_jobs": [...], // re-ranked based on the answer
  "reasoning": "string" // why the ranking changed
}
```

API requirements:

- Proper HTTP status codes and error handling
 - Input validation (e.g. empty resume, missing fields)
 - Response time should be reasonable — no unnecessary blocking calls
-

4. Submission Requirements

4a. Code (GitHub Repository)

- Public GitHub repo or a shared private link
- Must run locally with either:
 - `docker-compose up`, **or**
 - A clear README with setup steps (max 5 commands)
- Include a `.env.example` file for any required API keys
- Do not commit real API keys

4b. Technical Write-up

Include a `WRITEUP.md` in your repo (500–800 words) answering the following four questions:

1. Design Choices

Which embedding model did you choose and why? What alternatives did you consider and reject? What trade-offs did you make?

2. Agentic Architecture

Describe or diagram your agent's tool-calling flow. Why did you split it into two tool calls instead of one large prompt? What are the failure modes of your agent design?

3. Honest Weaknesses

What would break with noisy or poorly written resumes? What would break at scale (10,000 concurrent requests)? What corners did you cut due to time?

4. Next Steps

If you had two more days, what single improvement would have the highest impact and why?

5. Evaluation Rubric

Area	Weight	What We Look For
Semantic embeddings & ranking	20%	Correct cosine similarity, sensible model choice, embeddings used meaningfully
Agentic tool-call design	25%	Real tool calls with structured I/O, clean agent loop, not faked via prompt chaining
LLM prompt quality	20%	Precise instructions, structured outputs, good clarifying question quality
API correctness, code quality & deployment	20%	Clean FastAPI, error handling, readable code, live Vercel URL works
Technical write-up	15%	Honest, specific, shows depth of understanding — not just <i>what</i> but <i>why</i>
Bonus: <code>/refine</code> endpoint	+10%	Re-ranks correctly using the candidate's answer, reasoning is coherent

6. Constraints & Rules

- You may use **any** open-source libraries, LLM APIs, or frameworks
- You may **not** use a pre-built job recommendation library or end-to-end matching product
- The write-up must be your own — we will discuss it in the follow-up interview
- If you are genuinely blocked, note it in the write-up with your best guess at a solution — that is more valuable than silence

7. What We Are NOT Evaluating

- A frontend or UI
- Perfect recommendation accuracy (the dataset is small by design)
- Use of any specific LLM provider or cloud platform
- Fancy infrastructure (a simple local FastAPI is fine)

8. Deployment on Vercel (*Mandatory*)

Your FastAPI application must be deployed and publicly accessible on **Vercel**. A working live URL is required as part of your submission.

Steps to Deploy FastAPI on Vercel

1. Project structure

Vercel expects a specific layout for Python/FastAPI projects:

```
your-project/
├── api/
│   └── index.py      ← your FastAPI app entry point
├── jobs.json         ← the job dataset
├── requirements.txt
├── vercel.json       ← Vercel config (see below)
└── .env.example
```

Your `api/index.py` should export the FastAPI app as `app`:

```
from fastapi import FastAPI
# ... your full app code ...
app = FastAPI()
```

2. `vercel.json` configuration

Create this file at the root of your repo:

```
{
  "version": 2,
  "builds": [
```

```
{
  "src": "api/index.py",
  "use": "@vercel/python"
},
"routes": [
  {
    "src": "/(.*)",
    "dest": "api/index.py"
  }
]
}
```

3. Environment variables

Add your API keys (e.g. `OPENAI_API_KEY`) in the Vercel dashboard under **Project** → **Settings** → **Environment Variables**. Never hardcode keys in your source.

4. Deploy

```
npm i -g vercel      # install Vercel CLI
vercel login
vercel --prod        # deploy from project root
```

Vercel will output a public URL like `https://your-project.vercel.app`.

5. Verify your deployment

Test your live endpoints before submitting:

```
curl -X POST https://your-project.vercel.app/recommend \
-H "Content-Type: application/json" \
-d '{"resume_text": "Python developer with NLP experience..."}'
```


Vercel Constraints to Be Aware Of

Constraint	Detail
Execution timeout	Max 60 seconds per request on free tier — LLM + embedding calls must fit within this
No persistent disk	Do not write files at runtime; load <code>jobs.json</code> at startup or embed it in code
Cold starts	First request may be slow (~2–3s); mention this in your write-up
Memory limit	1024 MB on free tier — large local embedding models may not fit

Tip: If your embedding model is too large for Vercel's free tier, switch to an API-based embedding service (OpenAI, Cohere) rather than a locally loaded `sentence-transformers` model. Document this decision in your write-up.

9. Submission Instructions

1. Push your code to a GitHub repository
2. Deploy to Vercel and confirm the public URL is live
3. Share the repository link, your **live Vercel URL**, and your `WRITEUP.md`
4. Include a short Loom or screen recording (3–5 minutes) demoing the **live deployed API** — not just localhost — this is **mandatory**
5. Submit via ankit@cantileverlabs.com before your deadline

If you have any clarifying questions about the assignment, email ankit@cantileverlabs.com. Asking good questions is a skill we value.

10. Sample Resume for Testing

Use the resume of your choice to test your system during development:

Your `/recommend` endpoint should return meaningful, differentiated results for this resume. If all scores are clustered above 0.90, your embeddings are not working correctly.

We look forward to seeing what you build. Good luck.